Unidad 1

Almacenamiento información en ficheros

Ejercicios

Clases para la gestión de ficheros y directorios

Ventajas e inconvenientes de las distintas formas de acceso

Recuperar información almacenada en ficheros

Almacenar información en ficheros

Conversiones entre diferentes formatos de ficheros

Se han previsto y gestionado las excepciones

Se han probado y documentado las aplicaciones desarrolladas

Clases para la gestión de ficheros y directorios

Crear una utilidad que liste todos los archivos y subdirectorios de una ruta introducida por el usuario.

```
public void ej1(String ruta){
    Path dir = Paths.get(ruta);
    listarContenidoRecursivo(dir,0);
}

public void listarContenidoRecursivo(Path p,int t){
    File dir = new File(String.valueOf(p));
    if (dir.isDirectory()) {
        String[] archivos = dir.list();
        for (String archivo : archivos) {
            Path newP = Paths.get(p.toString() + "/" + archivo);
            String tabulacion = "";
            for (int i = 0; i < t ; i++){
                tabulacion = tabulacion.concat("\t");
            }
            System.out.println(tabulacion + archivo);
            listarContenidoRecursivo(newP,t+1);
        }
    }
}
```

Mostrar todos los ficheros .txt de un directorio y sus fechas de modificación.

```
public void ej2(String ruta) {
    File dir = new File(ruta);
    if (dir.isDirectory()) {
        String[] archivos = dir.list();
        try {
            for (String archivo: archivos) {
                if (archivo.endsWith(".txt")) {
                    System.out.println(archivo + " - " + Files.getLastModifiedTime((new File(ruta + "/" +
archivo)).toPath()));
                }
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Pedir al usuario una ruta y comprobar si es un fichero, un directorio o no existe.

```
public void ej3(){
    System.out.printf("Introduce la ruta: ");
    String ruta = new Scanner(System.in).nextLine();
    Path dir = Paths.get(ruta);
    if (!Files.exists(dir)){
        System.out.println("El archivo no existe");
    } else {
        String tipo ="";
        if (Files.isDirectory(dir)) {
            tipo = "directorio";
        }
        else {
            tipo = "archivo";
        }
        System.out.println("El archivo es un " + tipo);
    }
}
```

Crear un programa que cree un árbol de directorios a partir de una estructura definida por código.

```
public void ej4(String ruta){
    Path dir = Paths.get(ruta);
    try {
        System.out.println(Files.createDirectories(dir));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

Eliminar de forma recursiva todos los ficheros de un directorio que tengan más de 7 días.

```
public void ej5(String ruta){
    try {
        Path archivoAntiguo = Path.of("Directorio/Directorio 1.2/Directorio 1.2.1/Archivo Antigüo");
        Files.createFile(archivoAntiguo);
        Files.setLastModifiedTime(archivoAntiguo, FileTime.from(Instant.now().minus(999,ChronoUnit.DAYS)));
        System.out.println("Archivo antiguo creado");
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    Path dir = Paths.get(ruta);
    eliminarContenidoRecursivo(dir);
}

public void eliminarContenidoRecursivo(Path p){
    File dir = new File(String.valueOf(p));
    if (dir.isDirectory()) {
        String[] archivos = dir.list();
        for (String archivo : archivos) {
            Path newP = Paths.get(p.toString() + "/" + archivo);
            System.out.println("Comprobando "+ archivo);
            eliminarContenidoRecursivo(newP);
        }
    } else if (dir.isFile()){
        try {
            if (Files.getLastModifiedTime(p).toInstant().isBefore(Instant.now().minus(7, ChronoUnit.DAYS))) {
                dir.delete();
                System.out.println("Se ha eliminado el archivo " + dir.getAbsolutePath());
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Copiar todos los ficheros de un tipo específico de una carpeta a otra.

```
public void ej6(String ruta1, String ruta2, String type){
    System.out.println("---- COMIENZO EJERCICIO 6 ----");
    Path dir1 = Paths.get(ruta1);
    Path dir2 = Paths.get(ruta2);
    try {
        for (Path archivo : Files.list(dir1).toList()){
            if (archivo. toString().endsWith("." + type)){
                Files.copy(archivo,dir2.resolve(archivo.getFileName()));
            }
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 6 -----\n");
}
```

Comprobar el tamaño total de todos los ficheros en una carpeta y mostrarlo en MB.

```
public void ej7(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 7 ----");
    Path dir1 = Paths.get(ruta);
    try {
        for (Path archivo : Files.list(dir1).toList()){
            if (!Files.isDirectory(archivo)){
                System.out.println(archivo.getFileName() + " · " + Files.size(archivo)/2048 + "MB");
            }
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 7 -----\n");
}
```

Ventajas e inconvenientes de las distintas formas de acceso

Leer un fichero grande línea por línea (acceso secuencial) y contar cuántas líneas contienen una palabra clave.

```
public void ej1(String ruta, String clave){
    System.out.println("---- COMIENZO EJERCICIO 1 ----");
    Path dir = Paths.get(ruta);
    int cont = 0;
    try(BufferedReader br = Files.newBufferedReader(dir)) {
        String linea;
        while ((linea = br.readLine()) != null){
            if (linea.contains(clave)) cont++;
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("Palabra clave " + clave + " encontrada en " + cont + " líneas.");
    System.out.println("----- FIN DE EJERCICIO 1 -----\n");
}
```

Crear una clase Registro de tamaño fijo y guardarla en un fichero binario usando acceso aleatorio.

```
public static void ej2(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 2 ----");
    Path dir = Paths.get(ruta);
    List<Registro> rList = List.of(
            new Registro(1,2),
            new Registro(2,3),
            new Registro(3,4),
            new Registro(4,5),
            new Registro(5,6)
    );
    writeRecords(rList,dir);

    System.out.println("----- FIN DE EJERCICIO 2 -----\n");
}
private static void writeRecords(List<Registro> rList, Path dir){
    try (RandomAccessFile raf = new RandomAccessFile(dir.toFile(),"rw")) {
        for (Registro r : rList) {
            raf.writeInt(r.id());
            raf.writeInt(r.numero());
            System.out.println("se ha escrito el registro " + r.toString());
        }
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

Modificar un registro en una posición concreta de un fichero sin reescribir el fichero completo.

```
public static void ej3(String ruta,int id, int num){
    System.out.println("---- COMIENZO EJERCICIO 3 ----");
    Path dir = Paths.get(ruta);
    try (RandomAccessFile raf = new RandomAccessFile(dir.toFile(),"rw")) {
        raf.skipBytes((id*8)-4);
        raf.writeInt(num);
        System.out.println("Se ha modificado el registro " + id + " cambiando su numero a " + num);
        leerFichero(ruta);
    } catch (Exception e) {
        throw new RuntimeException(e);
   }
    System.out.println("----- FIN DE EJERCICIO 3 -----\n");
}
```

Comparar el tiempo de lectura secuencial vs aleatoria en un fichero con 10.000 registros de tamaño fijo.

Crear un programa que permita insertar, modificar y eliminar registros en un fichero binario con acceso aleatorio.

Escribir un índice con las posiciones de los registros en un fichero para acceder a ellos más rápidamente.

Recuperar información almacenada en ficheros

Leer un fichero de texto con datos de usuarios (nombre, edad, correo) y mostrarlos por consola.

```
public void ej1(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 1 ----");
    Path dir = Paths.get(ruta);
    try (BufferedReader br = Files.newBufferedReader(dir)) {
        for (String linea : br.readAllLines()){
                String[] registro = linea.split(",");
                System.out.println(registro[0]+" "+registro[1]+" "+registro[2]);
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 1 -----\n");
}
```

Leer un fichero CSV y cargar los datos en una lista de objetos.

```
public static void ej2(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 2 ----");
    ArrayList<Persona> personas = new ArrayList<>();
    Path dir = Paths.get(ruta);
    try (BufferedReader br = Files.newBufferedReader(dir)) {
        for (String linea : br.readAllLines()){
            String[] registro = linea.split(",");
            personas.add(new Persona(registro[0],Integer.parseInt(registro[1]),registro[2]));
        }
        System.out.println("Personas: ");
        System.out.println(personas.toString());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }

    System.out.println("----- FIN DE EJERCICIO 2 -----\n");
}
```

Leer datos binarios y convertirlos a un formato legible en consola.

```
public static void ej3(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 3 ----");
    Path dir = Paths.get(ruta);
    try(ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dir.toFile()));) {
        while (true) {
            Persona p = (Persona) ois.readObject();
            System.out.println(p);
        }
    } catch (EOFException e) {
    } catch (IOException e) {
        throw new RuntimeException(e);
    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 3 -----\n");
}
```

Leer un fichero de objetos serializados y mostrarlos ordenados por uno de sus atributos.

```
public static void ej4(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 4 ----");
    Path dir = Paths.get(ruta);
    ArrayList<Persona> personas = new ArrayList<>();
    try(ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dir.toFile()));) {
        while (true) {
            Persona p = (Persona) ois.readObject();
            personas.add(p);
        }
    } catch (EOFException e) {
    } catch (IOException e) {
        throw new RuntimeException(e);
    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    }
    personas.sort(Comparator.comparing(Persona::getEdad));
    for (Persona p : personas) System.out.println(p);
    System.out.println("----- FIN DE EJERCICIO 4 -----\n");
}
```

Almacenar información en ficheros

Guardar una lista de objetos Producto en un fichero de texto en formato CSV.

```
public static void ej1(String ruta, List<Producto> productList){
    System.out.println("---- COMIENZO EJERCICIO 1 ----");
    Path dir = Paths.get(ruta);
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(dir.toFile()))) {
        for (Producto p : productList) {
            bw.write(p.toString());
            bw.newLine();
            System.out.println("Escrito nueva linea en " + dir.getFileName() +": " + p.toString());
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 1 -----\n");
}
```

Guardar registros con DataOutputStream y asegurarte de que el formato sea legible al leer.

```
public static void ej2(String ruta, List<Producto> productList) {
    System.out.println("---- COMIENZO EJERCICIO 2 ----");
    Path dir = Paths.get(ruta);
    try(DataOutputStream dos = new DataOutputStream(Files.newOutputStream(dir))
    ) {
        for (Producto p : productList) {
            dos.writeInt(p.getId());
            dos.writeInt(p.getName().length());
            dos.writeChars(p.getName());
            dos.writeDouble(p.getPrice());
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try(DataInputStream dis = new DataInputStream(Files.newInputStream(dir))
    ) {
        System.out.println("Leyendo con data input stream los datos escritos: ");
        while (true) {
            int id = dis.readInt();
            int chars = dis.readInt();
            String nombre = "";
            int i;
            for (i = 0; i < chars; i++) {
                nombre = nombre.concat(String.valueOf(dis.readChar()));
            }
            double price = dis.readDouble();
            System.out.println(id + "," + nombre + "," + price);
        }
    } catch (EOFException e){
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 2 -----\n");
}
```

Serializar una lista de objetos Alumno y guardarla en un fichero .dat.

```
public static void ej3(String ruta, List<Alumno> alumnoList){
    System.out.println("---- COMIENZO EJERCICIO 3 ----");
    Path dir = Paths.get(ruta);
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(dir.toFile()))) {
        for (Alumno a : alumnoList){
            oos.writeObject(a);
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dir.toFile()))) {
        System.out.println("Leyendo los Alumnos escritos:");
        while (true){
            Alumno alumno = (Alumno) ois.readObject();
            System.out.println(alumno.toString());
        }
    } catch (EOFException e){
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
        throw new RuntimeException(e);
    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 3 -----\n");
}
```

Guardar información en un fichero binario pero con compresión usando GZIPOutputStream.

```
public static void ej4(String ruta, List<Producto> productList){
    System.out.println("---- COMIENZO EJERCICIO 4 ----");
    Path dir = Paths.get(ruta);
    try(DataOutputStream dos = new DataOutputStream(new GZIPOutputStream(Files.newOutputStream(dir)))
    ) {
        for (Producto p : productList) {
            dos.writeInt(p.getId());
            dos.writeInt(p.getName().length());
            dos.writeChars(p.getName());
            dos.writeDouble(p.getPrice());
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try(DataInputStream dis = new DataInputStream(Files.newInputStream(dir))
    ) {
        System.out.println("Leyendo con data input stream los datos escritos: ");
        while (true) {
            int id = dis.readInt();
            int chars = dis.readInt();
            String nombre = "";
            int i;
            for (i = 0; i < chars; i++) {
                nombre = nombre.concat(String.valueOf(dis.readChar()));
            }
            double price = dis.readDouble();
            System.out.println(id + "," + nombre + "," + price);
        }
    } catch (EOFException e){
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try(DataInputStream dis = new DataInputStream(new GZIPInputStream(Files.newInputStream(dir)))
    ) {
        System.out.println("\nLeyendo con GZIP input stream los datos escritos con GZIP: ");
        while (true) {
            int id = dis.readInt();
            int chars = dis.readInt();
            String nombre = "";
            int i;
            for (i = 0; i < chars; i++) {
                nombre = nombre.concat(String.valueOf(dis.readChar()));
            }
            double price = dis.readDouble();
            System.out.println(id + "," + nombre + "," + price);
        }
    } catch (EOFException e){
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 4 -----\n");
}
```

Conversiones entre diferentes formatos de ficheros

Convertir un fichero CSV de productos a un fichero binario.

```
public static void ej1(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 1 ----");
    Path dir = Paths.get(ruta);
    try (BufferedReader br = new BufferedReader(new FileReader(dir.toFile()));
         ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("Directorio/Apartado5/Productos.dat"))
    ) {
        List<String> productoList = br.readAllLines();
        ArrayList<Producto> productos = new ArrayList<>();
        for (String s : productoList){
            productos.add(new
Producto(Integer.parseInt(s.split(",")[0]),s.split(",")[1],Double.parseDouble(s.split(",")[2])));
        }
        for (Producto p : productos){
            oos.writeObject(p);
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("Directorio/Apartado5/Productos.dat"))) {
        System.out.println("Leyendo los productos escritos:");
        while (true){
            Producto producto = (Producto) ois.readObject();
            System.out.println(producto.toString());
        }
    } catch (EOFException e){
    } catch (ClassNotFoundException | IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 1 -----\n");
}
```

Convertir un fichero binario a un fichero de texto legible para exportación.

```
public static void ej2(String ruta) {
    System.out.println("---- COMIENZO EJERCICIO 2 ----");
    File dir = new File(ruta);
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dir))) {
        while (true){
            Producto producto = (Producto) ois.readObject();
            System.out.println(producto.toString());
        }
    } catch (EOFException e){
    } catch (ClassNotFoundException | IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 2 -----\n");
}
```

Cargar objetos desde un fichero .dat y exportarlos a CSV.

```
public static void ej3(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 3 ----");
    File dir = new File(ruta);
    ArrayList<Producto> productoArrayList = new ArrayList<>();
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dir))) {
        while (true){
            Producto producto = (Producto) ois.readObject();
            productoArrayList.add(producto);
        }
    } catch (EOFException e){
    } catch (ClassNotFoundException | IOException e) {
        throw new RuntimeException(e);
    }
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta.replace(".dat", "DatTo.csv")))) {
        for (Producto p : productoArrayList){
            bw.write(p.toString());
            bw.newLine();
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    try (BufferedReader br = new BufferedReader(new FileReader(ruta.replace(".dat", "DatTo.csv")))) {
        System.out.println("Leyendo el fichero csv creado:");
        for (String s : br.readAllLines()){
            System.out.println(s);
        }
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 3 -----\n");
```

Se han previsto y gestionado las excepciones

Crear una aplicación que intente leer un fichero que no existe y capture adecuadamente la excepción.

```
public static void ej1(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 1 ----");
    Path dir = Paths.get(ruta);
    try (BufferedReader br = new BufferedReader(new FileReader(dir.toFile()))) {
    } catch (FileNotFoundException e) {
        System.out.println("\u001B[31mEjercicio 1 Apartado 6: El archivo " + dir.getFileName() + " no
existe\u001B[0m");
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 1 -----\n");
}
```

Simular la apertura de un fichero protegido (sin permisos) y capturar SecurityException.

```
public static void ej2() {
    System.out.println("---- COMIENZO EJERCICIO 2 ----");
    try{
        throw new SecurityException();
    } catch (SecurityException e) {
        System.out.println("\u001B[31mEjercicio 2 Apartado 6: El archivo no es accesible\u001B[0m");
    }
    System.out.println("----- FIN DE EJERCICIO 2 -----\n");
}
```

Leer un fichero mal formateado y capturar NumberFormatException sin que se caiga el programa.

Leer objetos de un fichero .dat con fin de fichero controlado con EOFException.

```
public static void ej4(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 4 ----");
    Path dir = Paths.get(ruta);
    try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(dir))) {
        while (true){
            ois.readObject();
        }
    } catch (EOFException e) {
        System.out.println("\u001B[31mfin de archivo\u001B[0m");
    } catch (IOException | ClassNotFoundException e) {
        throw new RuntimeException(e);
```

```
    }
    System.out.println("----- FIN DE EJERCICIO 4 -----\n");
}
```

Crear un programa que intente escribir en una carpeta inexistente y capture el error.

```
public static void ej5(String ruta){
    System.out.println("---- COMIENZO EJERCICIO 5 ----");
    Path dir = Paths.get(ruta);
    try {
        if (!Files.exists(dir)) {
            throw new FileNotFoundException();
        } else {
            FileWriter fw = new FileWriter(dir.toString() + "/Sigma");
            fw.write("sigma");
            fw.close();
        }
    } catch (FileNotFoundException e) {
        System.out.println("\u001B[31mNo existe el directorio en el que se intenta escribir\u001B[0m");
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    System.out.println("----- FIN DE EJERCICIO 5 -----\n");
}
```

Se han probado y documentado las aplicaciones desarrolladas

Crear un programa de ejemplo que lea y escriba un fichero, documentar cada método con Javadoc.

El código es muy largo para ponerlo aquí. Está en src/Apartado7/Main

Probar qué ocurre al leer un fichero vacío y documentar el comportamiento esperado.

Muestra:

-------- Inicio Fichero --------

------- Final de Fichero -------